POLYNOMIAL REGRESSION (PR)

1. Polynomial Regression Nedir?

Polynomial Regression (Polinom Regresyon), **lineer regresyonun genişletilmiş bir versiyonudur**.
Temel fark:

- Lineer regresyon → düz çizgi (doğrusal ilişki)
- Polynomial regresyon → **eğri (nonlineer ilişki)**

☞ Ama önemli nokta:
Model **parametreler açısından lineerdir**, sadece **özellikler dönüştürülür**.

2. Modelin Matematiksel Gösterimi

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \cdots + \beta_d x^d$$

Burada:

- d → polinom derecesi
- β → model parametreleri
- x^2, x^3 → yeni oluşturulan özellikler

3. Modelin Temel Çalışma Mantığı

Polynomial Regression'ın mantığı:

1. Mevcut veride doğrusal olmayan ilişki varsa
2. Girdi değişkenlerini dönüştürür:
3. • $x \to x, x^2, x^3, \ldots$
4. Bu yeni veri üzerinde lineer regresyon uygular
5. Sonuç: **eğri bir model elde edilir.**

📌 Örnek:

- Lineer model: düz çizgi
- Polynomial model: parabola, kübik eğri vb.

4. Model Nasıl Öğrenir? (Training Süreci)

Adım adım süreç:
1. Feature Transformation
Veri şu hale getirilir:

$$X \to [1, x, x^2, x^3, \ldots, x^d]$$

2. Model Eğitimi
Amaç: hata minimize etmek

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

3. Optimizasyon

Parametreler şu yöntemlerle bulunur:

- Normal Equation
- Gradient Descent

4. Sonuç

Model:

- Veriye en iyi uyan eğriyi öğrenir
- Nonlineer ilişkileri yakalar

5. Hangi Problemlerde Kullanılır?

Polynomial Regression şu durumlarda tercih edilir:

✓ **Nonlineer ilişkiler varsa**
✓ **Küçük ve orta ölçekli veri setleri**
✓ **Fiziksel sistem modelleme**

6. Avantajları

✅ Basit ve anlaşılır.
✅ Lineer regresyon altyapısını kullanır.
✅ Nonlineer ilişkileri modelleyebilir.
✅ Hızlıdır.

7. Dezavantajları

❌ Overfitting riski yüksektir.

- Derece çok büyürse model ezber yapar.

❌ Extrapolation (dış tahmin) kötü olabilir.

- Veri dışına çıkınca saçma sonuçlar verebilir

❌ Yüksek derecelerde:

- Model kararsız olur
- Numerik problemler oluşabilir

❌ Çok boyutlu veride:

- Feature sayısı hızla artar

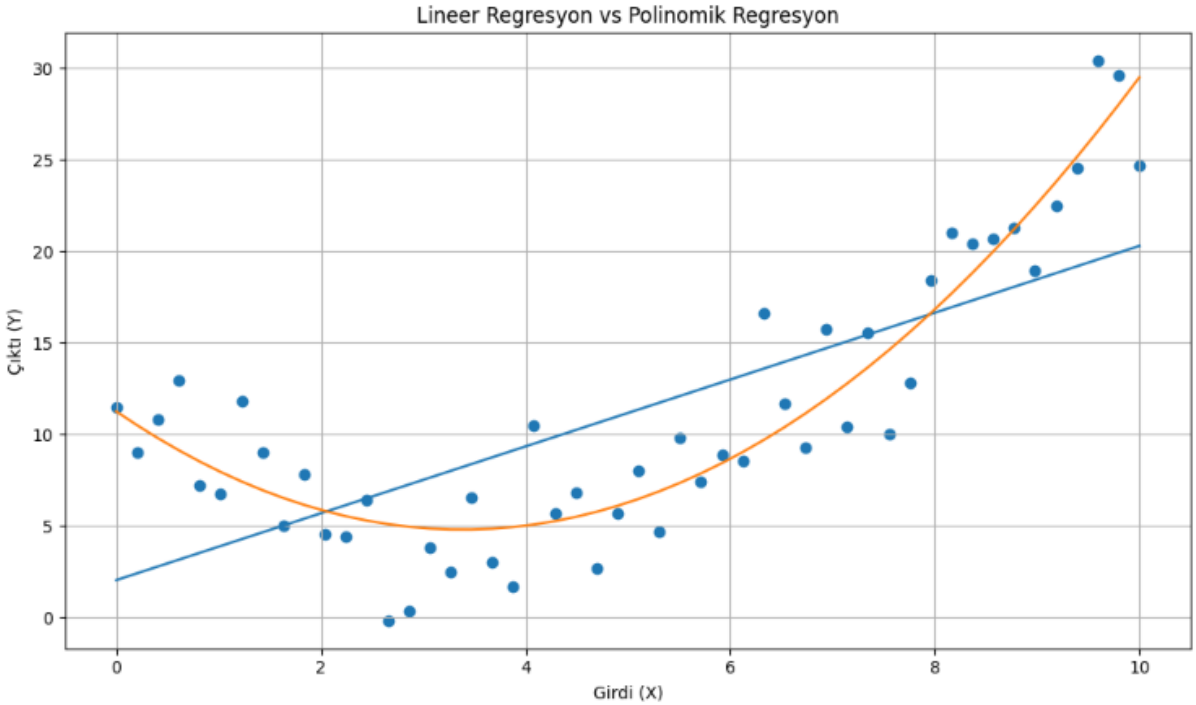
---

8. Overfitting Problemi

📌 Derece arttıkça:

- Eğitim hatası ↓
- Test hatası ↑

9. Özet


Lineer Regresyon vs Polinomik Regresyon
Çıktı (Y)
Girdi (X)
30
25
20
15
10
5
0
0
2
4
6
8
10

Support Vector Regression (SVR) Nedir?

Support Vector Regression (SVR), veriler arasındaki ilişkiyi öğrenerek **sayısal değer tahmini yapan bir makine öğrenmesi yöntemidir**.

☞ Temel fikir:

- Tüm noktaları tam olarak tahmin etmeye çalışmaz
- Bunun yerine:
  "Belirli bir hata toleransı içinde kalmayı" hedefler

📌 Yani:

- Küçük hataları görmezden gelir
- Büyük hataları cezalandırır

2. Modelin Temel Çalışma Mantığı

SVR şu soruya cevap arar:

☞ "Veriyi yaklaşık olarak temsil eden en düzgün (en basit) fonksiyon nedir?"

🎯 **ε (epsilon) mantığı**

Model bir **tolerans aralığı** tanımlar:

$$|y - f(x)| \leq \varepsilon$$

Bu ne demek?

- Tahmin ile gerçek değer arasındaki fark küçükse → sorun yok
- Bu aralığın dışına çıkarsa → ceza uygulanır

🎁 **Tüp (Tube) Analojisi**

SVR'yi şöyle düşünebilirsin:

- Verinin etrafına bir **tüp (boru)** çiziyoruz
- Bu tüpün genişliği = ε
- Amaç:
  - Mümkün olduğunca çok noktayı tüp içinde tutmak
  - Ama tüpü de gereksiz yere büyütmemek

3. Model Nasıl Öğrenir? (Training Süreci)

SVR eğitimi şu dengeyi kurar:

☞ **Model karmaşıklığı + hata dengesi**

Amaç fonksiyonu (basitleştirilmiş):

$$\min \frac{1}{2}||w||^2 + C\sum(\text{hata})$$

Süreç:

1. Model bir fonksiyon oluşturur:
2. $f(x) = wx + b$
3. ε içinde kalan hataları yok sayar
4. ε dışına çıkan hataları cezalandırır

5. En “düz” (en basit) fonksiyonu seçer

📌 **Support Vector Nedir?**

- Tüpün **dışında kalan kritik noktalar**
- Modeli belirleyen veri noktalarıdır

👉 Tüm veri değil, sadece önemli olanlar kullanılır

4. Kernel (Çok Önemli – Basit Anlatım)

Bazen veri doğrusal değildir.
SVR burada şunu yapar:
👉 Veriyi **başka bir uzaya taşır**

Örnek kernel’lar:

- Linear → düz ilişki
- Polynomial → eğrisel
- RBF (Gaussian) → çok güçlü nonlinear model

📌 Basit mantık:

- Veri karmaşıksa → kernel kullanılır
- Model, karmaşık ilişkileri öğrenebilir

5. Hangi Problemlerde Kullanılır?

SVR şu durumlarda çok güçlüdür:

✓ **Küçük veri setleri**
✓ **Nonlinear ilişkiler**
✓ **Gürültülü veri**

6. Avantajları

✅ Küçük veri setlerinde çok başarılı
✅ Overfitting’e karşı dayanıklı
✅ Nonlinear ilişkileri öğrenebilir
✅ Sadece önemli noktaları kullanır (support vectors)

7. Dezavantajları

❌ Parametre seçimi zor
❌ Büyük veri setlerinde yavaş
❌ Yorumlanabilirliği düşüktür
❌ Parametreler yanlış seçilirse performans düşer

8. Önemli Parametreler

C (Ceza Parametresi)

- Büyük → hataya tolerans az (overfitting riski)
- Küçük → daha esnek model

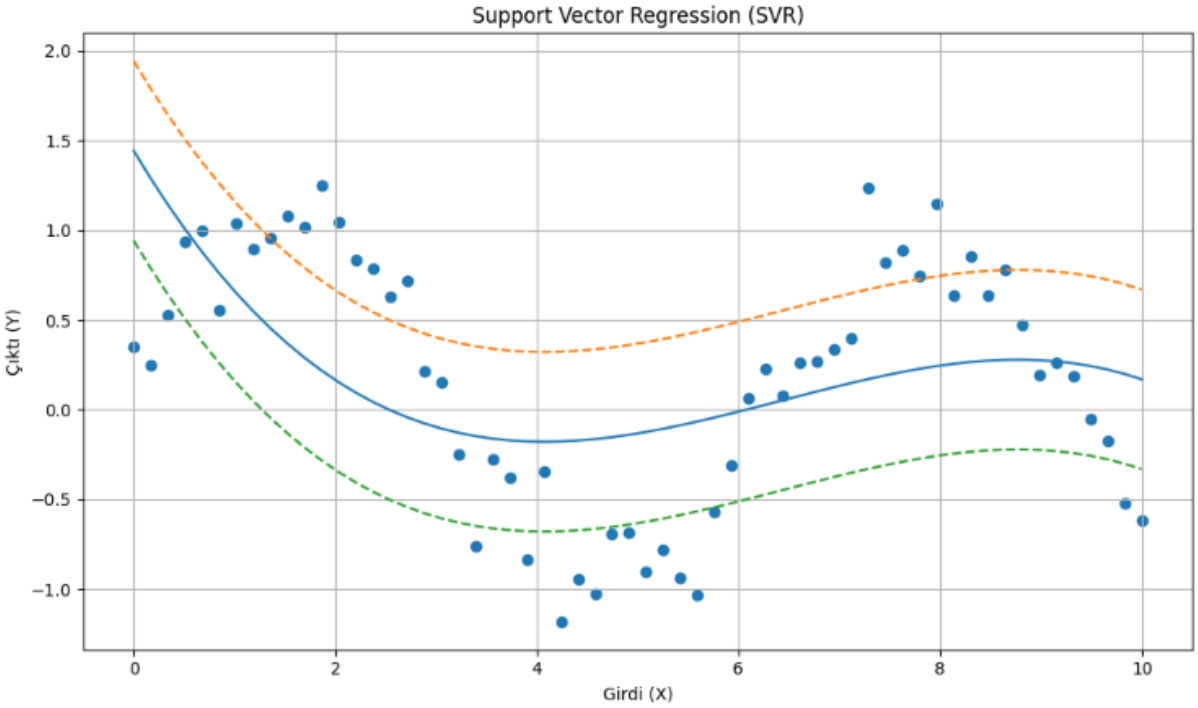
ε (Epsilon)

- Büyük → daha az hassas model
- Küçük → daha hassas model

Kernel

- Linear → basit veri
- RBF → karmaşık veri

9. Özet



Random Forest Regression (RF) Nedir?

Random Forest, **birden fazla karar ağacının birlikte çalıştığı (ensemble)** bir regresyon yöntemidir.

☞ Temel fikir:
"Tek bir karar ağacı yerine, birçok karar ağacı oluştur ve sonuçlarını birleştir."

📌 Yani:

- Decision Tree = tek ağaç 🌳
- Random Forest = ağaçlardan oluşan bir orman 🌳🌳🌳🌳

2. Modelin Temel Çalışma Mantığı

Random Forest şu şekilde çalışır:

1. Veri setinden **rastgele alt kümeler** seçilir
2. Her alt küme ile **ayrı bir karar ağacı** oluşturulur
3. Tüm ağaçların tahminleri alınır
4. **Ortalama alınarak final sonuç bulunur**

Matematiksel ifade:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^{N} T_i(x)$$

- $T_i(x)$ → i. ağacın tahmini
- N → ağaç sayısı

📌 Önemli nokta:
Her ağaç **farklı veri ve farklı özelliklerle** eğitilir → çeşitlilik oluşur

3. Model Nasıl Öğrenir? (Training Süreci)
Random Forest öğrenme süreci 3 ana adımdan oluşur:

◆ **Bootstrap Sampling (Veri Örnekleme)**

- Veri setinden **rastgele seçimlerle** alt veri setleri oluşturulur
- Aynı veri birden fazla kez seçilebilir

☞ Buna **bagging (bootstrap aggregating)** denir

◆ **Random Feature Selection**

- Her split (bölme) sırasında:
  - Tüm özellikler değil
  - **Rastgele seçilen özellikler kullanılır**

☞ Bu, ağaçların birbirine benzemesini engeller

◆ **Ağaçların Eğitilmesi**

- Her alt veri ile bir Decision Tree eğitilir
- Ağaçlar genelde:
  - Derin (deep)
  - Budama yapılmadan (overfit olabilir)

◆ **Tahmin (Prediction)**

- Tüm ağaçların çıktıları alınır
- Ortalama alınır → final tahmin

4. Hangi Problemlerde Kullanılır?

Random Forest şu durumlarda çok etkilidir:

✓ **Nonlinear veri**

- Karmaşık ilişkileri öğrenebilir

✓ **Gürültülü veri**

- Tek ağaçtan daha dayanıklıdır

✓ **Orta ve büyük veri setleri**

✓ **Feature sayısı fazla olan problemler**

5. Avantajları

✅ Overfitting'e karşı dayanıklı

- Tek ağaç yerine ortalama alır

✅ Yüksek doğruluk

- Genelde güçlü performans

✅ Feature importance verebilir

- Hangi değişken önemli?

✅ Nonlinear ilişkileri öğrenir

6. Dezavantajları

❌ Yorumlanması zor

- Tek ağaç gibi anlaşılır değil

❌ Hesaplama maliyeti yüksek

- Çok ağaç → daha fazla zaman

❌ Çok büyük veri setlerinde yavaş olabilir

❌ Model boyutu büyük olabilir

7. Decision Tree ile Karşılaştırma

Özellik	**Decision Tree**	**Random Forest**
Model	Tek ağaç	Çoklu ağaç
Overfitting	Yüksek	Düşük
Doğruluk	Orta	Yüksek
Yorumlanabilirlik	Kolay	Zor

8. Önemli Parametreler

◆ **n_estimators**

- Ağaç sayısı
- ↑ arttıkça doğruluk ↑ (ama maliyet de ↑)

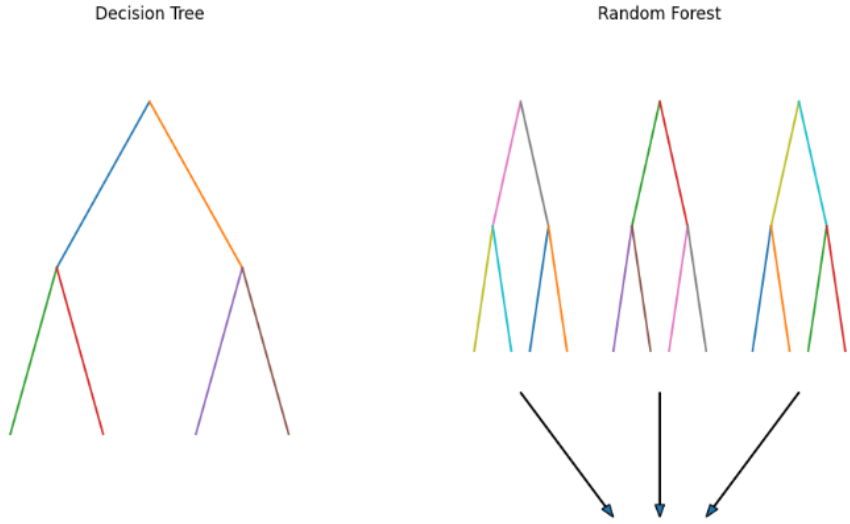
◆ **max_depth**

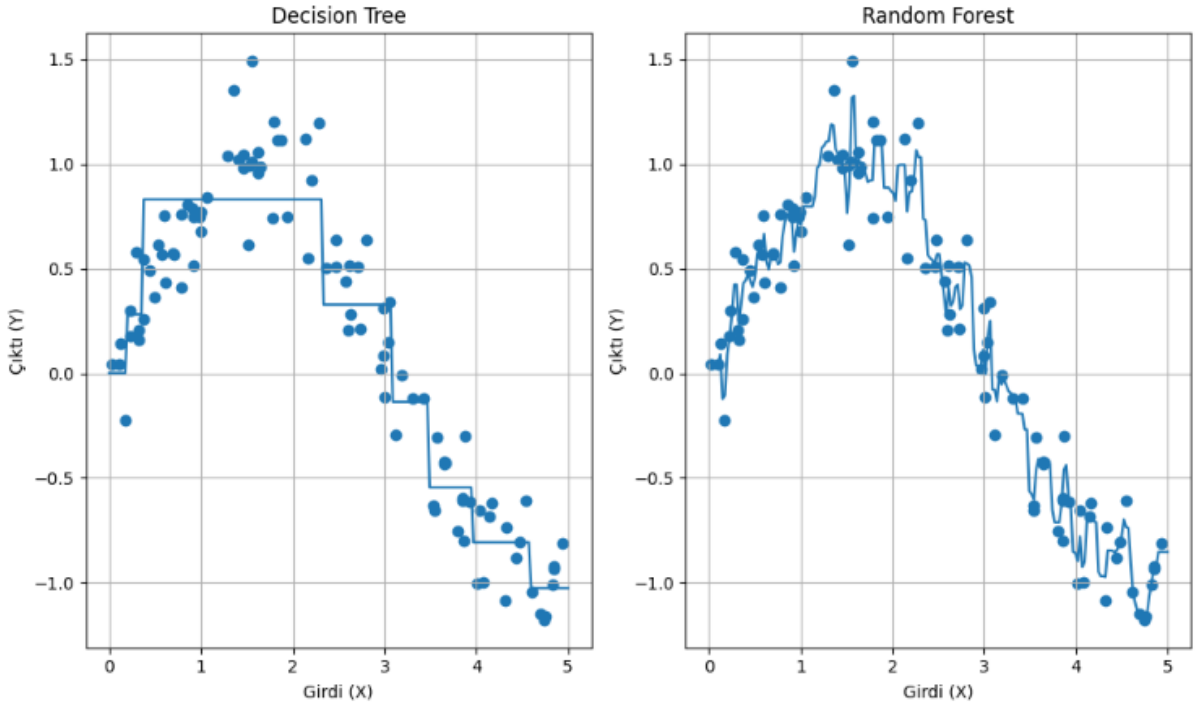
- Ağacın derinliği
- Çok büyük → overfitting

◆ **max_features**

- Her split'te kullanılacak feature sayısı

9. Özet





Gradient Boosting Regression (GB) Nedir?

Gradient Boosting, **birden fazla karar ağacını ardışık (sıralı) şekilde eğiterek hataları azaltan** bir regresyon yöntemidir.

☞ Temel fikir:
"Her yeni model, bir önceki modelin yaptığı hataları düzeltmeye çalışır."

📌 Yani:

- Decision Tree → tek model
- Random Forest → paralel çok model
- Gradient Boosting → **sırayla öğrenen modeller**

2. Modelin Temel Çalışma Mantığı

Gradient Boosting şu şekilde çalışır:

1. İlk model oluşturulur (genelde basit bir ağaç)
2. Tahmin hataları hesaplanır
3. Yeni model → bu hataları öğrenir
4. Tüm modeller toplanır → final tahmin

Matematiksel ifade:

$$F(x) = \sum_{m=1}^{M} \gamma_m h_m(x)$$

- $h_m(x)$ → m. ağaç
- γ_m → ağırlık
- M → model sayısı

📌 Önemli:

- Her model küçük katkı yapar
- Ama birlikte güçlü bir model oluşur

3. Model Nasıl Öğrenir? (Training Süreci)

Gradient Boosting'in öğrenme süreci **iteratif (adım adım)** ilerler:

Adım 1: Başlangıç modeli

- Basit bir tahmin yapılır (örneğin ortalama değer)

Adım 2: Hata hesaplama

$$r_i = y_i - \hat{y}_i$$

☞ Bu hatalara **residual (artık)** denir

Adım 3: Yeni model eğitme

- Yeni karar ağacı:
  ☞ Bu hataları öğrenir

Adım 4: Model güncelleme

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

- $\eta \rightarrow$ learning rate

Adım 5: Tekrar

- Bu işlem birçok kez yapılır
- Her adımda model daha iyi hale gelir

4. Gradient (Neden bu isim?)

Gradient Boosting ismi şuradan gelir:

👉 Model hatayı **gradient (türev)** kullanarak minimize eder

📌 Basitçe:

"Model, hatanın yönünü bulur ve o yönde kendini düzeltir."

5. Hangi Problemlerde Kullanılır?

Gradient Boosting çok güçlüdür ve şu durumlarda tercih edilir:

✅ **Karmaşık (nonlinear) veri**

- Çok iyi öğrenir

✅ **Yüksek doğruluk gereken problemler**

- Kaggle yarışmaları

✅ **Orta ve büyük veri setleri**

✅ **Tabular veri (en güçlü olduğu alan)**

6. Avantajları

✅ Çok yüksek doğruluk

✅ Nonlinear ilişkileri öğrenir

✅ Overfitting kontrol edilebilir

✅ Feature importance verir

7. Dezavantajları

❌ Eğitim süresi uzun

❌ Parametre ayarı zor

❌ Overfitting riski (yanlış ayarlarda)

❌ Paralel çalışması zor (sıralı yapı)

8. Random Forest ile Farkı

Özellik	Random Forest	Gradient Boosting
Eğitim	Paralel	Sıralı
Amaç	Ortalama almak	Hata düzeltmek
Overfitting	Düşük	Kontrol edilmezse yüksek
Doğruluk	Yüksek	Çok yüksek

9. Önemli Parametreler

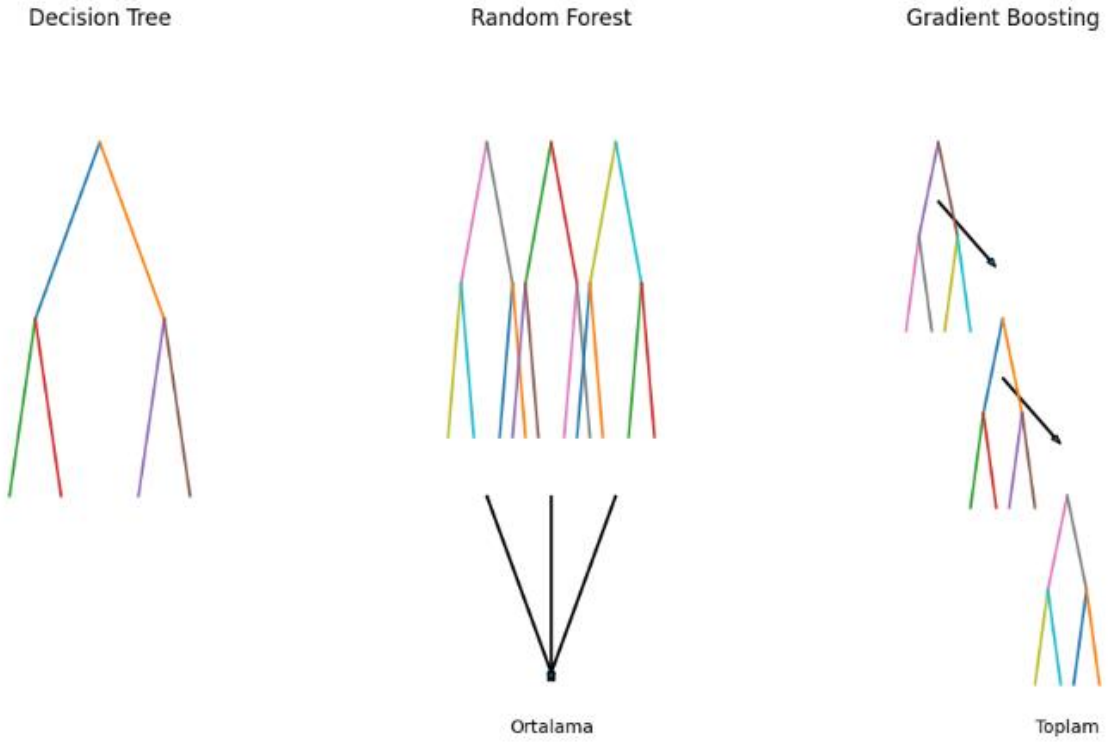
◆ **n_estimators**

- Model sayısı

◆ **learning_rate (η)**

- Her modelin katkısı
- Küçük → daha stabil ama yavaş

◆ **max_depth**

- Ağaç derinliği

10. Özet